

Shibboleth SSO 検証環境 構築手順書（純 Windows 版）

第 0.9 版

2026-07-09

0. 改訂履歴

版	日付	変更概要	備考
0.1	2026-07-05	純 Windows 版として新規作成。\$1～\$4（目的・前提・全体アーキテクチャ・パラメータ・ロードマップ）を記載	WSL 版 v0.15 を土台に、顧客 PLM 検証環境の再現用として起こす
0.2	2026-07-05	フェーズ 1（検証環境の初期設定：前提確認・時刻同期・着手前チェックポイント）を追記。推奨アカウント値（わかりやすさ優先）を \$3 に追記	入れ子仮想化は不要のため簡素化
0.3	2026-07-05	フェーズ 2（hosts・内部 CA・idp/sp サーバ証明書(a)の発行・PFX 化・信頼登録）を追記。証明書作成は Git for Windows 同梱の openssl を使用	ネットワーク方式選定は不要（同一 Windows・127.0.0.1）
0.4	2026-07-05	フェーズ 3（ApacheDS 導入・パーティション作成・ou=people・テストユーザー 01PLM01/02・検索バインド idp-reader）を追記	LDAP は ApacheDS。GUI（Directory Studio）+ LDIF
0.5	2026-07-08	フェーズ 3 の実機知見を反映（Java 前倒し／zip 優先方針／C:統一／ApacheDS は LDAP ポート 10389・設定は ou=config 方式で config.ldif 無し→既定パーティション dc=example,dc=com 流用／Directory Studio は exe 起動／userPassword は SSHA 自動ハッシュ）。\$6.4.1「証明書コマンドの解説」を追記	実機検証の反映と解説の追加
0.6	2026-07-08	フェーズ 4（OpenJDK は \$5.6 で導入済み。Tomcat 10.1 の zip 展開・環境変数・service.bat によるサービス化・8080 起動確認）を追記	Tomcat は zip + service.bat
0.7	2026-07-08	フェーズ 5（Shibboleth IdP 5：zip 展開 + install.bat 導入・context フラグメント配置・ldap.properties（10389/dc=example,dc=com/idp-reader）・emailAddress NameID の元 mail・起動確認）を追記。フェーズ 4 のサービス	本構成の山場

版	日付	変更概要	備考
0.8	2026-07-09	ス表示名を実機値「Apache Tomcat 10.1 Tomcat10」に補足 フェーズ5実機反映：\$9.3を訂正し JSTL (API+Glassfish 実装の2 jar) 追加+build.bat 再ビルドを必須手順 に (未追加だと /idp/profile/status が ClassNotFoundException: jakarta.servlet.jsp.jstl.core.Config-ServletException)。\$9.4に trustCertificates/trustStore のコメントアウト、saml-nameid の bean 重複回避を追記	JSTL は必須だった
0.9	2026-07-09	フェーズ6 (Tomcat 直 HTTPS 8443 公開: idp.pfx を conf に配置し server.xml に SSLHostConfig+Certificate の 8443 コネクタを追加、8080 は localhost 限定、鍵マーク確認) を追記。Apache 前段は不要	WSL 版の Apache+mirrored を Tomcat コネクタ1つで代替

本書は、WSL 版構築手順書 (WSL2 上に IdP スタックを構築した学習環境) を土台に、**WSL を一切使わない純 Windows 構成**で顧客 PLM 検証環境を再現するための手順書です。SP 側 (IIS+Shibboleth SP) と SAML 設計の考え方は WSL 版から流用し、IdP 側 (Tomcat/Shibboleth IdP) ・LDAP を Windows ネイティブに置き換えています。

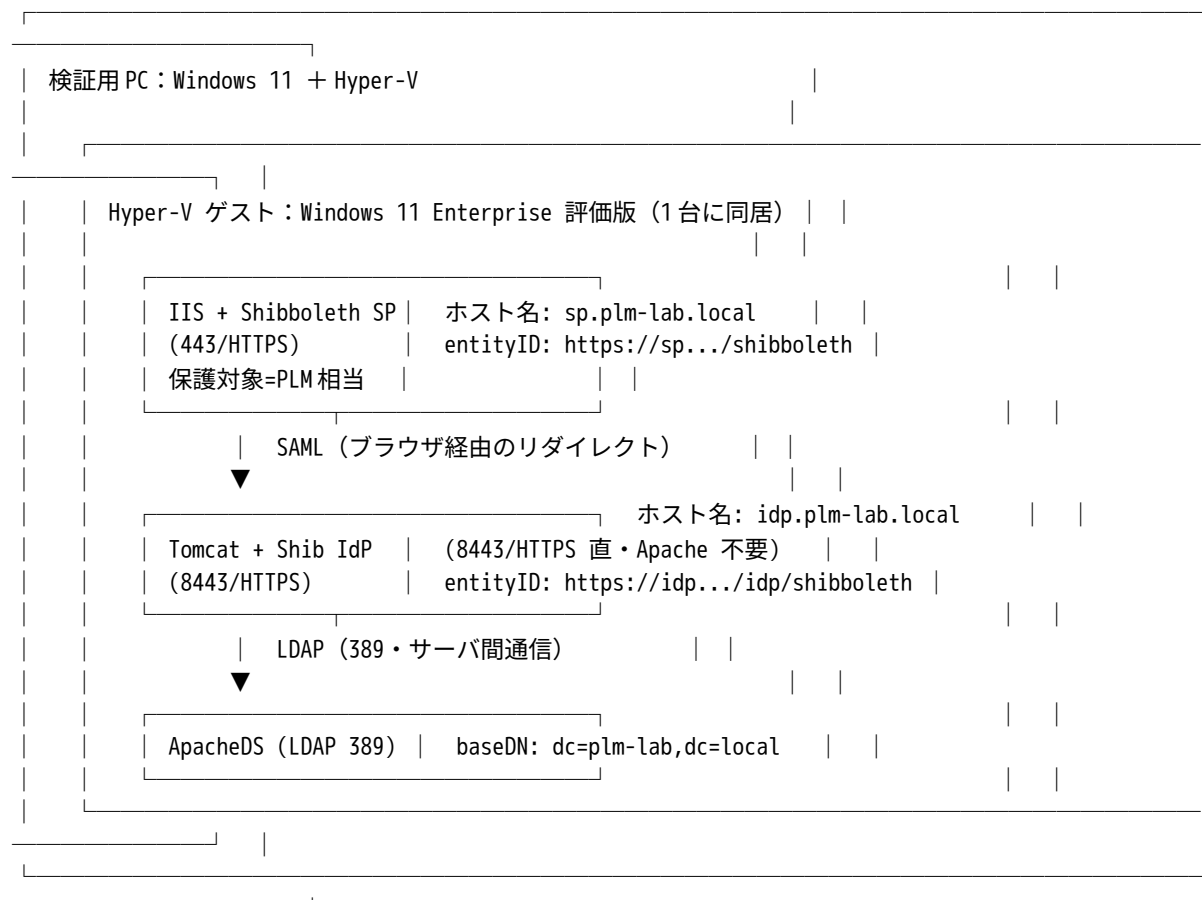
1. 目的と前提

- 目的**：顧客 PLM 環境 (Shibboleth=SP、前段=IIS、PLM は C:\inetpub\wwwroot 配下の IIS アプリ、IdP=Entra ID/独自認証) に近い SAML SSO 検証環境を、**全メンバーが習熟している Windows のみ**で構築し、SP 側の設定と動作を確認する。
- 方針**：コンテナや WSL を使わず、**すべて Windows 上**で手作業構築する。IdP は学習用に自前構築し、将来 Entra ID に差し替え可能な形にする。
- 物理ホスト**：検証用 PC (Windows 11、Hyper-V 有効)。
- 検証用 Windows**：Hyper-V 仮想マシン上の **Windows 11 Enterprise 評価版** (90 日 /slmgr /rearm で延長。付録 A)。
- 役割分担 (すべて同一のゲスト Windows 上に同居)**：
 - IIS + Shibboleth SP**：ブラウザからの入口 (前段)。未認証を IdP へ、認証後は REMOTE_USER を確定し、wwwroot 配下の PLM 相当アプリを実行。
 - Tomcat + Shibboleth IdP**：学習用テスト IdP (HTTPS 8443 を直接公開)。将来 Entra ID に差し替え可能。
 - ApacheDS (LDAP)**：ユーザーディレクトリ。IdP が認証・属性取得のために参照。
- 前提 (電源管理)**：物理ホスト・ゲスト仮想マシンとも **Windows のスリープ/休止は無効化**していること (組織の対象 PC も同様の運用)。本書はこの前提に立ち、スリープ復帰に伴う時刻ずれ対策は主手順から除外する (参考手順は付録 C)。
- 顧客環境との対応 (確認済み事項)**：
 - 前段=IIS、SP=Shibboleth SP (IIS ネイティブモジュール)、PLM=IIS 上のアプリ (顧客と一致)。

- IdP=Entra ID（+独自認証）。識別子は **emailAddress 形式**（例 01PLM01@plm-lab.local）。
- PLM は将来、メール形式の識別子を受け取り @ の前を切り出して従来の識別番号として扱う想定（**PLM アプリ側の責務**であり、本書の SP/IdP 構築範囲外）。

⚠ 重要な前提：本環境は Windows 仮想マシン 1 台にすべてが同居します。仮想マシンを削除すると IdP/LDAP/SP すべてが失われます。評価版の期限は削除・再インストールではなく `slmgr /rearm` で延長してください（付録 A）。作業成果の保全・再構築検証の考え方は付録 B を参照。

2. 全体アーキテクチャ



SAML Web SSO の流れ（未認証時）：

1. ブラウザが IIS（SP・443）の保護対象（PLM 相当）にアクセスする。
2. SP が「未認証」と判断し、ブラウザを IdP（8443）へリダイレクトする。
3. ブラウザが IdP（Tomcat・8443）のログイン画面にアクセスする。
4. IdP が ApacheDS（LDAP・389）に問い合わせてユーザーを認証する（サーバ間通信）。
5. IdP が SAML アサーションを、ブラウザ経由で SP の ACS へ POST する。
6. SP がアサーションを検証してセッションを確立し、REMOTE_USER を確定する。
7. IIS が wwwroot 配下の PLM 相当アプリを実行して応答する。

WSL 版との違い：IdP 側が Windows ネイティブのため、**IdP 前段の Apache HTTPD は不要**（Tomcat が HTTPS 8443 を直接公開）。WSL2 の mirrored ネットワーク・localhost 転送・

ポート衝突といった問題は発生しない。IIS (SP) と Tomcat (IdP) は同一 Windows 上に同居するが、**互いに直接は通信せず**、ブラウザのリダイレクトと事前交換したメタデータで連携する。LDAP を参照するのは IdP のみ。

3. パラメーター一覧（確定・調整用）

本書全体で参照する値。**着手前に内容を確認し**、変更する場合はここを起点に各フェーズへ反映してください。

項目	既定値	備考
内部ドメイン	plm-lab.local	検証用の架空ドメイン
IdP ホスト名	idp.plm-lab.local	Tomcat (IdP) 側
SP ホスト名	sp.plm-lab.local	IIS (SP) 側
Java	OpenJDK 17 (Windows 版)	Shibboleth IdP 5 の要件。Temurin 等
Servlet コンテナ	Apache Tomcat 10.1.x (Windows サービス)	IdP 5 は Tomcat 10.1 必須 (9 系不可)
IdP	Shibboleth IdP 5.x	Java 17 / Jakarta 名前空間
SP	Shibboleth SP 3.x (IIS ネイティブモジュール)	IIS 用
LDAP	ApacheDS (Java 製・Windows サービス)	Apache Directory Studio で投入。inetOrgPerson
IdP entityID	https://idp.plm-lab.local/idp/shibboleth	フェーズ 5 で確定
SP entityID	https://sp.plm-lab.local/shibboleth	フェーズ 8 で確定
LDAP ベース DN	dc=plm-lab,dc=local	ApacheDS (フェーズ 3)
LDAP 検索バインド	uid=idp-reader,ou=people,dc=plm-lab,dc=local	IdP の LDAP 検索用 (読取専用)
テストユーザー	uid=01PLM01 / uid=01PLM02	ou=people 配下。識別番号 (社内テスト環境の形式に準拠)
テストユーザーの mail	01PLM01@plm-lab.local / 01PLM02@plm-lab.local	emailAddress 形式 NameID の元 (顧客 Entra の形式に準拠)
NameID 形式	emailAddress	顧客 (Entra ID) の形式に合わせる (WSL 版の unspecified から変更)
REMOTE_USER	優先リスト (例 azure_id … gid_id に倣う)	メール形式 NameID を載せる。学習では 1~2 個で可
IdP ホーム	C:\opt\shibboleth-idp	Shibboleth IdP 5 (Windows・フェーズ 5)
Tomcat ホーム	C:\opt\tomcat (例)	Windows サービスとして稼働 (フェーズ 4)

項目	既定値	備考
SP インストール先	C:\opt\shibboleth-sp	既定。C:\inetpub 配下は不可（鍵・設定の漏洩リスク）
PLM アプリ配置	C:\inetpub\wwwroot 配下	検証では REMOTE_USER 表示のテストページ
SP ポート（HTTPS）	443	IIS（SP）
IdP ポート（HTTPS）	8443	Tomcat（IdP）直公開。同一 Windows 上での 443 衝突回避
内部 CA ルート名	PLM-Lab Root CA	フェーズ 2 で作成。idp/sp のサーバ証明書を発行
証明書作成ツール	openssl for Windows（Git for Windows 同梱等）	WSL 版の openssl 手順を流用
ゲスト VM メモリ	8 GB（目安）	フルスタック同居のため
ゲスト VM vCPU	4	目安
タイムゾーン	Asia/Tokyo	ゲスト Windows

ホスト名は当面 hosts ファイルで解決します（DNS サーバは立てません）。SP・IdP とともにゲスト Windows 上のため、127.0.0.1 で解決させます（フェーズ 2）。

2 種類の証明書（再掲・重要）：(a) TLS/HTTPS サーバ証明書（ブラウザに信頼される・SAN 一致必須。内部 CA で idp/sp を発行）と、(b) SAML 署名・暗号化証明書（メタデータ交換で信頼・自己署名可・CA 信頼もホスト名一致も不要。IdP はインストール時に自動生成、SP は keygen で生成）。準備が要るのは (a)。詳細はフェーズ 2 で扱う。

3.1 アカウント・パスワード一覧（わかりやすさ優先）

本検証環境では**わかりやすさを優先**し、既定値や「Joe アカウント（uid とパスワードが同じ）」を採用する。**いずれも検証専用であり、実際の環境構築時にはセキュリティを考慮して強固な値に変更すること**。OS ユーザーとしての専用アカウント（WSL 版の tomcat 相当）は Windows では作らず、各サービスは Windows のサービスアカウント（Local System 等）で動作する。構築作業はローカル管理者（本書では Administrator。読み替え可）で、昇格した PowerShell／コマンドプロンプトで行う。

#	アカウント／鍵	値（検証用）	使用箇所	フェーズ
1	ApacheDS 管理者	uid=admin,ou=system / secret	ApacheDS 管理（既定値のまま）	3
2	テストユーザー 1	uid=01PLM01 / パスワード 01PLM01	ログイン検証（Joe アカウント）	3
3	テストユーザー 2	uid=01PLM02 / パスワード 01PLM02	2 人目・再現性検証（Joe アカウント）	3
4	IdP 検索バインド	uid=idp-reader / パスワード idp-reader	IdP の LDAP 検索（読取専用）	3・5
5	IdP キーストア	changeit	IdP install.bat 対	5

#	アカウント／鍵	値（検証用）	使用箇所	フェーズ
6	IdP Sealer	changeit	IdP install.bat 対話	5
7	TLS 証明書 PFX (idp/sp)	changeit	idp.pfx / sp.pfx の作成・取込 (Tomcat・IIS)	2・6・7

OS ユーザーの扱い：構築は Administrator（ローカル管理者）。Tomcat/IdP・ApacheDS・shibd（Shibboleth Daemon）はいずれも Windows サービスとして Local System 等のサービスアカウントで動作するため、Linux の tomcat ユーザーのような専用 OS ユーザーの作成・所有権付与は不要。

4. 構築フェーズ全体像（ロードマップ）

フェーズ	内容	WSL 版との関係	本書での状態
1	検証環境の初期設定 (ゲスト Windows VM・時刻同期・前提)	新規 (WSL 導入を廃し簡素化)	✓ 本版で記載
2	ネットワーク土台 (hosts・内部 CA・idp/sp 証明書 (a)・ポート設計)	変更 (openssl for Windows で流用)	✓ 本版で記載
3	ApacheDS (LDAP) 導入・ディレクトリ設計・テスト ユーザー投入	新規 (OpenLDAP から置換)	✓ 本版で記載
4	OpenJDK + Tomcat (Windows サービス)	変更 (Windows 版・service.bat)	✓ 本版で記載
5	Shibboleth IdP 5 (Windows・LDAP 連携・emailAddressNameID 準備)	変更 (install.bat・Windows パス)	✓ 本版で記載

フェーズ	内容	WSL 版との関係	本書での状態
6	Tomcat 直 H TTPS (844 3) 公開	置換 (Apache 前段を廃止)	☑ 本版で記載
7	IIS (SP の 保護対象サ イト・443/ TLS)	流用 (WSL 版とほぼ同一)	■ 未
8	Shibboleth S P (IIS ネイ ティブモ ジュール・ サイト全体 保護)	流用 (WSL 版とほぼ同一)	■ 未
9	メタデータ 交換 (IdP → SP の相 互信頼・初 回 SSO)	変更 (:8443 補正が不要に)	■ 未
10	属性連携 (emailAdd ress 形式 N ameID を R EMOTE_USE R に載せ る)	変更 (unspecified→emailAddress)	■ 未
11	結合テスト (ログイ ン・再現 性・再起動 堅牢性・ロ グ)	流用／変更	■ 未

各フェーズは「目的 → 前提 → 手順 → 動作確認」の順で記載します。付録：A (評価版 rearm) / B (バックアップ・再現性検証のチェックポイント運用) / C (スリープ環境向け時刻再同期) / D (トラブルシュート・Windows 固有) / E (オフライン導入)。

5. フェーズ 1：検証環境の初期設定

目的：Hyper-V ゲストの Windows 11 を、以降の全スタック (ApacheDS / Tomcat + IdP / IIS + SP) を同居させる土台として整える。あわせて、SAML でつまづきやすい**時刻同期**を最初に固め、やり直しに備えた**着手前チェックポイント**を取得する。

WSL 版との違い：本構成は WSL2 を使わないため、**入れ子 (ネステッド) 仮想化の有効化は不要**。ゲスト Windows 上で Java/Tomcat/IIS が動くだけなので、通常の Hyper-V ゲスト

として扱える（WSL 版 \$5.2 の ExposeVirtualizationExtensions や MAC スプーフィングは本版では不要）。

5.1 事前確認

- ・ 検証用 PC で Hyper-V が有効で、ゲストの Windows 11 評価版が作成済みであること。
- ・ ゲストに割り当てるメモリ（8GB 目安）・vCPU（4 目安）の余裕が検証用 PC にあること。
- ・ ゲスト Windows にローカル管理者（Administrator。自宅では読み替え）でサインインできること。
- ・ 物理ホスト・ゲストとも Windows のスリープ／休止が無効であること（\$1 の前提）。

5.2 ゲスト VM の電源オプションと着手前チェックポイント

やり直しに備え、素の状態のチェックポイントを取得します。まず、電源断時に保存状態へ入らずシャットダウンさせる設定にしてから、VM を停止した状態でチェックポイントを取得します。検証用 PC（物理ホスト）の管理者権限 PowerShell で実行します。

1) 対象 VM 名を確認

Get-VM

2) 電源断時に保存状態にせずシャットダウンさせる（保存状態からの復帰による時刻ずれ回避）

Set-VM -Name "<VM 名>" -AutomaticStopAction ShutDown

3) 種類を標準（Standard）チェックポイントに（本構成は入れ子仮想化を使わないため Standard で可）

Set-VM -Name "<VM 名>" -CheckpointType Standard

4) ゲストをシャットダウン（停止状態で取得するのが最も安全）

Stop-VM -Name "<VM 名>"

5) 着手前の素の状態を取得

Checkpoint-VM -Name "<VM 名>" -SnapshotName "Phase1 前_素の Windows11"

⚠ パラメータ名の注意（-Name と -VMName）：VM 自体を操作する系（Set-VM／Get-VM／Start-VM／Stop-VM／Checkpoint-VM）は VM 名を -Name で指定する。VM の構成要素を操作する系（Set-VMProcessor／Set-VMemory 等）は -VMName だが、本フェーズでは使用しない。取り違えを避けたい場合は Get-VM "<VM 名>" | Set-VM -CheckpointType Standard のようにパイプで渡す。

チェックポイントは短期のやり直し用でありバックアップではありません（付録 B）。本構成は入れ子仮想化を使わないため、標準（Standard）チェックポイントが利用でき、稼働中の取得も比較的安全ですが、着手前の素の状態は**停止して**取得するのが確実です。

5.3 時刻同期の確認

SAML は IdP と SP の時刻の一致に敏感で、ズレると「clock skew」エラーの原因になります。本構成では IdP（Tomcat）も SP（IIS）も**同一のゲスト Windows 上**にあるため、両者の時刻は常に一致し、WSL 版のような OS 間の相対ずれは原理的に発生しません。したがって、ゲスト Windows の時計が正しく保たれていれば十分です。

- ・ **Hyper-V 統合サービス「時刻同期」が有効**であることを確認（既定で有効）。これによりゲスト Windows は検証用 PC（ホスト）の時計に追従します。ゲストの管理者 PowerShell で確認：

タイムゾーンが東京か

Get-TimeZone

東京でなければ設定

```
Set-TimeZone -Name "Tokyo Standard Time"
```

Windows Time サービスが稼働しているか (オンライン時は NTP 同期も有効)

```
w32tm /query /status
```

オンライン環境なら Windows Time サービス (w32tm) が NTP に同期します。オフライン環境でも、IdP と SP が同一 OS 上のため相対ずれは発生せず、SAML の clock skew は問題になりません。スリープを使う環境で運用する場合の復帰時再同期は付録 C を参照。

5.4 導入方針と作業用フォルダの準備

導入方針 (本書全体) : zip パッケージが選べるものは zip を展開して導入し、環境変数など必要な設定は手作業で行う (どこに何が入ったかが明確になり、再現・撤去・切り分けが容易)。導入物は原則 C:\opt 配下・空白を含まないパスに統一する (Java アプリでの空白起因トラブルを避ける)。作業は昇格した PowerShell / コマンドプロンプトで行う。

例外: ApacheDS 本体は zip 提供が無いためインストーラ (.exe) で導入する (§7.2)。

作業用フォルダを用意します。

```
New-Item -ItemType Directory -Force -Path C:\opt, C:\lab, C:\lab\ca, C:\lab\installers | Out-Null
```

- C:\opt: jdk-17 / directory-studio / tomcat / shibboleth-idp / shibboleth-sp の導入先。
- C:\lab\ca: 内部 CA・証明書の作成場所 (フェーズ 2)。
- C:\lab\installers: 入手した各インストーラ / zip の保管。

5.5 動作確認チェックリスト (フェーズ 1)

#	確認内容	コマンド / 方法	期待結果
1	ゲストにローカル管理者でサインイン	—	Administrator (読み替え可) でサインイン済み
2	タイムゾーン	Get-TimeZone	Tokyo Standard Time
3	時刻サービス	w32tm /query /status	稼働 (オンライン時は NTP 同期)
4	着手前チェックポイント	Hyper-V マネージャー	Phase1 前_素の Windows11 が存在
5	昇格実行の確認	管理者 PowerShell が使える	昇格プロセスでコマンド実行可

5.6 OpenJDK (Temurin 17) の導入 (ApacheDS・Tomcat の前提)

ApacheDS (フェーズ 3) と Tomcat (フェーズ 4) はいずれも Java で動作する。**Java は両者の共通基盤のため、ここ (フェーズ 3 の前) で先に導入しておく** (ApacheDS インストーラが JAVA_HOME を求めるため、順序として Java が先)。

1. **入手:** Adoptium (<https://adoptium.net/>) から **Temurin 17 (LTS)** ・ **Windows x64** ・ **JDK** ・ **zip** を入手し C:\lab\installers へ。
2. **展開&リネーム** (管理者 PowerShell) :

```
Expand-Archive -Path "C:\lab\installers\OpenJDK17U-jdk_x64_windows_hotspot_*.zip" -DestinationPath "C:\opt" -Force
Get-ChildItem C:\opt | Where-Object Name -like "jdk-17*"           # 展開フォルダ名を確認
Rename-Item "C:\opt\jdk-17.0.xx+xx" "C:\opt\jdk-17"             # 分かりやすく (任意)
```

3. 環境変数（システム）を手動設定：

```
[Environment]::SetEnvironmentVariable("JAVA_HOME", "C:\opt\jdk-17", "Machine")
$P = [Environment]::GetEnvironmentVariable("Path", "Machine")
[Environment]::SetEnvironmentVariable("Path", "$P;C:\opt\jdk-17\bin", "Machine")
```

設定後は**新しい PowerShell を開き直す**（既存セッションには反映されない）。

4. 確認：java -version（openjdk version “17...”）、echo \$env:JAVA_HOME。

この後にフェーズ2（証明書）・フェーズ3（ApacheDS）へ進む。ApacheDS インストーラが JAVA_HOME を尋ねたら C:\opt\jdk-17 を指定する。

すべて確認できれば、フェーズ1は完了です。次はフェーズ2（ネットワーク土台・内部CAとidp/sp証明書の発行）です。

6. フェーズ2：ネットワーク土台（hosts・内部CA・idp/spサーバ証明書）

目的：コンポーネント導入の前に、全体が依存する「名前解決（hosts）」と「TLS証明書」を先に確定する。純 Windows 構成では SP（IIS）も IdP（Tomcat）も**同一のゲスト Windows 上**にあり、いずれも 127.0.0.1 で解決するため、WSL 版のようなネットワーク方式の選定（NAT／mirrored の段階移行）や、localhost 転送・ポート転送に関する検討は**不要**になる。

WSL 版との違い：WSL 版では「ブラウザ→WSL2 の到達性」を確保するため mirrored モードや (A)→(B) 段階移行を検討したが、本構成では IdP が Windows ネイティブのため、その検討は丸ごと不要。ポート設計（SP=443／IdP=8443）だけは同じ考え方で踏襲する（同一ホスト上で 443 を取り合わないため）。

6.1 ポート設計

同一のゲスト Windows 上で SP（IIS）と IdP（Tomcat）が 443 を取り合わないよう、IdP を別ポート（8443）にします。

役割	ホスト名	URL（ベース）	ポート	バインド先
SP	sp.plm-lab.local	https://sp.plm-lab.local/	443	IIS
IdP	idp.plm-lab.local	https://idp.plm-lab.local:8443/idp/	8443	Tomcat（直 HTTPS）

SAML のエンドポイントはすべて URL なので、ポートが異なっても問題ありません。顧客の本番（IdP=Entra ID）では IdP は Entra 側の URL になるため、この 8443 は「学習用テスト IdP を同一ホストに同居させるための便宜」です。

6.2 名前解決（hosts）

DNS サーバは立てず、hosts で解決します。SP・IdP とも同一のゲスト Windows 上のため、両方を 127.0.0.1 に向けます。

ゲスト Windows : C:\Windows\System32\drivers\etc\hosts (管理者権限で編集。昇格したメモ帳等)

```
127.0.0.1 sp.plm-lab.local
127.0.0.1 idp.plm-lab.local
```

管理者 PowerShell で追記する場合：

```
Add-Content -Path "$env:SystemRoot\System32\drivers\etc\hosts" -Value "127.0.0.1`ts`sp.plm-lab.local`r`n127.0.0.1`ti`dp.plm-lab.local"
```

確認：

```
ping sp.plm-lab.local      # 127.0.0.1 に解決されること
ping idp.plm-lab.local    # 127.0.0.1 に解決されること
```

6.3 【参考】この構成で登場する 2 種類の証明書

Shibboleth では性格の異なる 2 種類の証明書が登場し、混同しやすいポイントです。役割を分けて理解してください。

観点	(a) TLS/HTTPS サーバ証明書	(b) SAML 署名・暗号化証明書
目的	ブラウザ↔サーバの HTTPS 通信の暗号化とサーバ真正性	SAML メッセージ（アサーション等）の署名・暗号化
使う場所	IIS (SP, 443)、Tomcat (IdP, 8443)	IdP・SP の SAML 処理内部
誰が検証するか	ブラウザ	相手方の IdP/SP
信頼の確立方法	CA を信頼ストアに登録 (PKI)	メタデータ交換で公開鍵を相互登録
ホスト名 (SAN) 一致	必須	不要
CA 署名	必要 (本書では内部 CA で発行)	不要 (自己署名が通常)
準備するタイミング	フェーズ 2 (本節)	各コンポーネント導入時に生成 (IdP=フェーズ 5、SP=フェーズ 8)

本節 (6.4) で用意するのは (a) のみ。(b) は後続フェーズでコンポーネントが生成します。両者は独立で、(b) に CA 信頼やホスト名一致は不要です。

6.4 内部 CA の作成と idp/sp サーバ証明書の発行 ((a) の準備)

Git for Windows 同梱の openssl を使います。**Git Bash** を起動して以下を実行します (WSL 版と同じ openssl 手順をそのまま流用)。作業場所は \$5.4 で作成した C:\lab\ca (Git Bash では /c/lab/ca)。

```
cd /c/lab/ca
```

1) 内部CAルート（秘密鍵と自己署名証明書、10年）

```
openssl genrsa -out rootCA.key 4096
openssl req -x509 -new -nodes -key rootCA.key -sha256 -days 3650 \
  -subj "//C=JP\O=PLM-Lab\CN=PLM-Lab Root CA" -out rootCA.crt
```

2) idp のサーバ証明書

```
cat > idp.ext <<'EOF'
subjectAltName = DNS:idp.plm-lab.local
extendedKeyUsage = serverAuth
EOF
openssl genrsa -out idp.key 2048
openssl req -new -key idp.key -subj "//C=JP\O=PLM-Lab\CN=idp.plm-lab.local" -out idp.csr
openssl x509 -req -in idp.csr -CA rootCA.crt -CAkey rootCA.key -CAcreateserial \
  -out idp.crt -days 825 -sha256 -extfile idp.ext
```

3) sp のサーバ証明書

```
cat > sp.ext <<'EOF'
subjectAltName = DNS:sp.plm-lab.local
extendedKeyUsage = serverAuth
EOF
openssl genrsa -out sp.key 2048
openssl req -new -key sp.key -subj "//C=JP\O=PLM-Lab\CN=sp.plm-lab.local" -out sp.csr
openssl x509 -req -in sp.csr -CA rootCA.crt -CAkey rootCA.key -CAcreateserial \
  -out sp.crt -days 825 -sha256 -extfile sp.ext
```

⚠ **Git Bash 特有の注意（MSYS のパス変換）**：Git Bash では -subj "//C=JP/..." の先頭の / が Windows パスに誤変換されることがあります。上記のように **先頭を // にし、区切りを ** ("//C=JP\O=PLM-Lab\CN=...") とすると回避できます。うまくいかない場合は、環境変数 MSYS_NO_PATHCONV=1 を付けて実行（例：MSYS_NO_PATHCONV=1 openssl req ...）してもよいです。

PFX (PKCS12) 化：IIS (sp) と Tomcat (idp) は、HTTPS バインドに証明書＋秘密鍵を **PFX 形式** で取り込みます。両方を作成します（エクスポートパスワードは §3.1 のとおり changeit）。

IIS (フェーズ7) 用

```
openssl pkcs12 -export -out sp.pfx -inkey sp.key -in sp.crt -certfile rootCA.crt -passout pass:changeit
```

Tomcat/IdP (フェーズ6) 用

```
openssl pkcs12 -export -out idp.pfx -inkey idp.key -in idp.crt -certfile rootCA.crt -passout pass:changeit
```

6.4.1 【解説】証明書コマンドの意味

§6.4 の各コマンドが「何をしているか」を、後から理解・説明できるよう解説します。全体像は「**自前の認証局（CA）を1つ作り、そのCAでidpとspのサーバ証明書に署名する**」というPKIの縮小版です。

3ステップの関係

- [1] 内部CAルートを作る (rootCA.key / rootCA.crt) … 署名する側（親）
 - └─[2] idp のサーバ証明書をCAに署名してもらう (idp.key / idp.crt)
 - └─[3] sp のサーバ証明書をCAに署名してもらう (sp.key / sp.crt)

サーバ証明書 (idp.crt/sp.crt) は単体では信頼されず、**信頼されたCA (rootCA) の署名が付いて初めてブラウザに信頼される**。だから先にCAを作り、そのCAで各サーバ証明書を署名する。

[1] 内部 CA ルート

- openssl genrsa -out rootCA.key 4096：CA の**秘密鍵**を生成。4096 は鍵長（CA は大元なのでサーバ証明書の 2048 より長く強度を確保）。生成物 rootCA.key は**最重要機密**（これで任意の証明書に署名できる）。
- openssl req -x509 -new -nodes -key rootCA.key -sha256 -days 3650 -subj "...CN=PLM-Lab Root CA" -out rootCA.crt：秘密鍵を使って CA 自身の**ルート証明書**を作る（**自己署名**）。
 - -x509：CSR でなく証明書そのものを出力（自己署名になる）。-new：新規。-nodes：秘密鍵をパスフレーズで暗号化しない（起動時のパスワード入力を不要にする）。-key：署名に使う秘密鍵。-sha256：署名ハッシュ。-days 3650：有効期間約 10 年（CA は長寿命）。-subj：サブジェクト（C 国／O 組織／CN＝この CA の名前。Windows の信頼ストアに表示される名前になる）。先頭 // と区切り \ は Git Bash のパス誤変換回避。
 - 生成物 rootCA.crt：CA の公開ルート証明書（**公開可**）。各マシンの信頼ストアに登録して「この CA が署名した証明書を信頼する」状態を作る。

[2] idp のサーバ証明書

- cat > idp.ext ...：証明書に付ける**拡張**を書いたファイル。subjectAltName = DNS:idp.plm-lab.local（**SAN**。現代のブラウザは CN でなく SAN でホスト名一致を判定するため必須）、extendedKeyUsage = serverAuth（**サーバ認証用途**）。
- openssl genrsa -out idp.key 2048：idp サーバの**秘密鍵**（サーバ用は 2048）。
- openssl req -new -key idp.key -subj "...CN=idp.plm-lab.local" -out idp.csr：CSR（**証明書署名要求＝申請書**）を作成。CSR には idp の**公開鍵**とサブジェクトが入る（この時点では CA 署名なし）。
- openssl x509 -req -in idp.csr -CA rootCA.crt -CAkey rootCA.key -CAcreateserial -out idp.crt -days 825 -sha256 -extfile idp.ext：CA が**申請書に署名して正式なサーバ証明書を発行**（中核）。
 - x509 -req：入力 CSR で、署名して証明書を出すモード。-CA/-CAkey：署名に使う CA 証明書と CA 秘密鍵。-CAcreateserial：シリアル番号管理ファイル（rootCA.srl）を作成/更新。-days 825：有効期間（ブラウザ制限に合わせ短め）。-extfile idp.ext：SAN・用途を証明書に付与（**忘れるとホスト名不一致警告**）。
 - 生成物 idp.crt：idp の公開鍵＋サブジェクト＋SAN＋**CA の署名**が入った正式なサーバ証明書。

[3] sp のサーバ証明書

[2] の idp を sp に置き換えただけで意味は同じ（SAN=DNS:sp.plm-lab.local、CN=sp.plm-lab.local）。**同じ CA（rootCA）で署名**するため、rootCA を 1 つ信頼登録すれば idp・sp 両方が信頼される。

生成ファイルまとめ

ファイル	種類	機密性	用途
rootCA.key	CA 秘密鍵	最重要機密	各サーバ証明書への署名
rootCA.crt	CA ルート証明書	公開可	信頼の起点。各マシンの信頼ストアに登録
rootCA.srl	シリアル管理	—	証明書ごとの通

ファイル	種類	機密性	用途
			し番号（自動）
idp.key / sp.key	サーバ秘密鍵	機密	Tomcat(IdP)・IIS (SP) の TLS 復号
idp.csr / sp.csr	署名要求	—	発行時の中間ファイル
idp.ext / sp.ext	拡張設定	—	SAN・用途を付与する中間ファイル
idp.crt / sp.crt	サーバ証明書	公開可	Tomcat(IdP,8443)・IIS(SP,443) の HTTPS
idp.pfx / sp.pfx	証明書+秘密鍵(PKCS12)	機密（鍵を含む）	Tomcat・IIS へのインポート用

要点：CA を自前で1つ作り rootCA を各マシンに登録すれば、内部のサーバ証明書がすべて信頼される。手順は「秘密鍵 → CSR - CA 署名 → 証明書」で、公的証明書取得の縮小版。SAN が無いと鍵マークにならないため subjectAltName が肝。鍵長は CA=4096 / サーバ=2048 と使い分ける。ここで作るのは (a) TLS/HTTPS サーバ証明書のみで、(b) SAML 署名・暗号化証明書はフェーズ 5・8 でコンポーネントが生成する（別物）。

6.5 rootCA をゲスト Windows の信頼ルートに登録

ブラウザ（Edge/Chrome は Windows 証明書ストアを使用）で鍵マークにするため、内部 CA ルートを「信頼されたルート証明機関」に登録します。管理者 PowerShell で：

```
Import-Certificate -FilePath "C:\lab\ca\rootCA.crt" -CertStoreLocation Cert:\LocalMachine\Root
```

別マシン（検証用 PC=ホストや LAN 上の PC）のブラウザからアクセスする「発展編」を行う場合は、そのマシンの証明書ストアにも同じ rootCA.crt を登録すれば、ホスト名にひも付く証明書はそのまま使えます（作り直し不要）。ただし本書の基本構成はゲスト Windows 上のブラウザで検証します。

6.6 動作確認チェックリスト（フェーズ 2）

#	確認内容	方法	期待結果
1	hosts 解決	ping idp.plm-lab.local / ping sp.plm-lab.local	いずれも 127.0.0.1
2	CA・証明書生成	ls /c/lab/ca (Git Bash)	rootCA.crt / idp.crt / sp.crt / idp.pfx / sp.pfx 等が存在
3	SAN の確認	openssl x509 -in idp.crt -noout -text \ grep -A1 "Subject Alternative"	DNS:idp.plm-lab.local
4	証明書の検証	openssl verify -CAfile rootCA.crt idp.crt / sp.crt	OK
5	Windows の CA 信頼	certlm.msc - 「信頼されたルート証	PLM-Lab Root CA が存在

#	確認内容	方法	期待結果
		明機関」	
	この時点では証明書を使う Web サーバ（Tomcat/IIS）がまだ無いため、HTTPS 応答の確認は各コンポーネント導入後（フェーズ 6・7）に行います。		

7. フェーズ 3：ApacheDS（LDAP ユーザーディレクトリ）

目的：IdP が認証・属性取得を行うユーザーの元データを、**ApacheDS**（Java 製 LDAP サーバ・Windows で動作）に用意する。ディレクトリ設計（baseDN・ou=people・テストユーザー・検索バインド）は WSL 版の OpenLDAP と同じ考え方を踏襲し、製品を ApacheDS に置き換える。ユーザー投入は **Apache Directory Studio**（GUI）または **LDIF インポート**で行う。

WSL 版との違い：OpenLDAP（apt・slapd・dpkg-reconfigure・LDIF+ldapadd）から、ApacheDS（Windows インストーラ・GUI ツール）に置き換わる。設計値（dc=plm-lab,dc=local／ou=people／inetOrgPerson／uid・mail・userPassword）は同じ。メール形式の識別子に対応するため、各ユーザーに **mail 属性**（01PLM01@plm-lab.local）を持たせる点が新しい。

7.1 ディレクトリ設計

dc=example,dc=com	← ベース DN（ApacheDS 既定パーティションを流用）
└── ou=people	← ユーザー・検索アカウント
└── uid=01PLM01（inetOrgPerson）	← テストユーザー 1（mail: 01PLM01@plm-lab.local）
└── uid=01PLM02（inetOrgPerson）	← テストユーザー 2（mail: 01PLM02@plm-lab.local）
└── uid=idp-reader（inetOrgPerson）	← IdP の検索バインド用（読取）
└── （将来）ou=groups	← ロール等（本書では未使用）

項目	値
パーティション suffix（baseDN）	dc=example,dc=com（ApacheDS 既定パーティションを流用。7.4 参照）
ユーザー OU	ou=people,dc=example,dc=com
テストユーザー	uid=01PLM01 / uid=01PLM02（objectClass: inetOrgPerson）。パスワード=uid（joe）
テストユーザーの mail	01PLM01@plm-lab.local / 01PLM02@plm-lab.local（emailAddress NameID の元）
検索バインド	uid=idp-reader,ou=people,dc=example,dc=com（パスワード idp-reader・読取専用）
ApacheDS 管理者	uid=admin,ou=system（既定パスワード secret）
LDAP ポート	10389 （ApacheDS 既定。IdP 側は ldap://localhost:10389 で参照）

baseDN について（実機の判断）：ApacheDS のインストーラ版（2.0.0.AM 系）は設定を config.ldif ではなく LDAP 内部（ou=config）で保持し、dc=plm-lab,dc=local パーティションの手作業追加は難度が高い（7.4 参照）。本書では**既定パーティション dc=example,dc=com**をそのまま流用する（学習・SSO 動作には影響しない）。WSL 版の dc=plm-lab,dc=local か

らの読み替え表は7.4に示す。mailは@plm-lab.localのまま（メールアドレスとbaseDNは別物なので揃える必要はない）。

ポートの注意：ApacheDSのLDAPポートは実機で**10389**（非特権ポート）。Directory StudioのNew ConnectionではPort入力欄の既定表示が**389**になることがあるが、実機の待受は10389なので**10389に合わせる**（netstat -ano | findstr 10389で確認）。WSL版は389だったが、本書では10389を用い、フェーズ5のIdP設定（ldapURL）もldap://localhost:10389とする。

7.2 ApacheDS のインストール

前提（Javaを先に導入）：ApacheDSはJavaで動作し、インストーラの途中で**JAVA_HOME**の入力を求められる。そのため**OpenJDK（Temurin 17）をApacheDSより前に導入**しておく（\$5.6参照）。インストーラでJAVA_HOMEを聞かれたら、JDKのルート（例C:\opt\jdk-17。直下にbin\java.exeがあるフォルダ。binは含めない）を指定する。

1. Apache Directoryの公式サイト（<https://directory.apache.org/apacheds/download/download-windows.html>）から、**ApacheDSのWindowsインストーラ（.exe・最新安定版）**を入手し、C:\lab\installersに保存。※ApacheDS本体はインストーラ（.exe）で導入する（zip提供が無いため。zip優先方針の例外）。
2. インストーラを**管理者として実行**。JAVA_HOMEに上記JDKを指定し、他は既定のまま進める（実機の導入先はC:\Program Files (x86)\ApacheDS、インスタンスは...\instances\default）。導入すると**ApacheDSのWindowsサービス**（既定インスタンス名default）が登録される。
3. サービスを起動・確認：

```
Get-Service | Where-Object { $_.DisplayName -like "*ApacheDS*" }
Start-Service <サービス名> # 停止していれば開始（サービス名は環境で異なる）
# LDAP 待受ポートの確認（実機は 10389）
netstat -ano | findstr 10389
```

実機ではLDAPは**10389**で待ち受ける（0.0.0.0:10389 LISTENING）。

7.3 Apache Directory Studio の導入と接続

1. 公式サイト（<https://directory.apache.org/studio/download/download-windows.html>）から**Apache Directory Studio（Windows版・zip）**を入手し、C:\opt\directory-studioに展開する（本書のzip優先方針）。
2. **起動：**zip展開では**インストールされず、Windowsスタートメニューにも登録されない**。エクスプローラーまたはコマンドからC:\opt\directory-studio\ApacheDirectoryStudio.exeを実行して起動する（よく使う場合はショートカットを作成）。
 - Javaが見つからず起動しない場合は、C:\opt\directory-studio\ApacheDirectoryStudio.iniに-vmとC:\opt\jdk-17\bin\javaw.exeの2行を追記する（通常は不要。JAVA_HOME設定済みのため）。
3. 起動後、**LDAP接続を作成：**メニュー**LDAP - New Connection**。
 - Network Parameter：Hostname localhost / Port **10389**（入力欄の既定表示が389の場合は10389に変更） / Encryption method **No encryption**（同一ホスト通信のため平文で可）
 - Connection name（任意）：管理作業用なのでplm-lab-adminなど用途が分かる名前を推奨
 - 次画面（Authentication）：Bind DN uid=admin,ou=system / Bind password secret
 - **Check Authentication**を押し、successfulを確認してから**Finish**。接続が緑色になれば成功。

4. 左の **LDAP Browser** で、既定パーティション `dc=example,dc=com` や `ou=system・ou=config` が見えることを確認。

7.4 パーティション (baseDN) の方針：既定パーティション `dc=example,dc=com` を流用

当初は WSL 版に合わせ `dc=plm-lab,dc=local` パーティションを新規作成する想定だったが、**実機のインストーラ版 ApacheDS (2.0.0.AM 系)** では手作業での新規作成は現実的でないことが判明した。理由：

- このバージョンは設定を `config.ldif` ファイルではなく **LDAP 内部 (ou=config)** として保持する (実際 `C:\Program Files (x86)\ApacheDS\instances\default\` 配下に `config.ldif` は存在せず、`partitions\` に `example・system` の DB がある)。パーティション追加は停止→設定エントリ (`ads-partition...` 階層) を正確に追記→再起動が必要で、書式ミスで**サービスが起動不能になるリスク**がある。
- Directory Studio に ApacheDS を「サーバ」として登録していない (接続のみ) ため、**LDAP Servers ビューの Open Configuration によるパーティション編集が使えない**。

そこで本書は、**ApacheDS の既定パーティション `dc=example,dc=com` をそのまま流用**する。パーティション作成は不要で、その配下に `ou=people` とユーザーを作る。学習・SSO 動作には一切影響しない。

baseDN 読み替え表 (当初の `plm-lab` 版 → 本書の既定流用版)

項目	当初 (WSL 版踏襲)	本書 (既定流用)
baseDN	<code>dc=plm-lab,dc=local</code>	<code>dc=example,dc=com</code>
ユーザー OU	<code>ou=people,dc=plm-lab,dc=local</code>	<code>ou=people,dc=example,dc=com</code>
テストユーザー	<code>uid=01PLM01,ou=people,dc=plm-lab,dc=local</code>	<code>uid=01PLM01,ou=people,dc=example,dc=com</code>
検索バインド	<code>uid=idp-reader,ou=people,dc=plm-lab,dc=local</code>	<code>uid=idp-reader,ou=people,dc=example,dc=com</code>
mail (変更なし)	<code>01PLM01@plm-lab.local</code>	<code>01PLM01@plm-lab.local</code>

どうしても `dc=plm-lab,dc=local` を作りたい場合は、ApacheDS の設定 (`ou=config` の `ou=partitions`) にパーティション定義を追加し、コンテキストエントリ (`dc=plm-lab,dc=local`) を投入する必要がある (バージョン依存・要サービス再起動)。本書では扱わない。

7.5 OU・テストユーザー・検索バインドの投入 (LDIF)

Directory Studio で **LDAP メニュー → New LDIF File** を開き、以下を貼り付けて、エディタ**右上の緑の ▶ (Execute LDIF)** で実行します (対象接続は管理者の `plm-lab-admin`)。 `dc=example,dc=com` 自体は既存のため LDIF に含めず、その配下だけ作ります。パスワードは分かりやすさ優先の平文で記載します (ApacheDS が格納時に自動で SSHA ハッシュ化するため、WSL 版の `slappasswd` に相当する作業は不要)。

```
# ユーザー OU (既存の dc=example,dc=com 配下に作成)
dn: ou=people,dc=example,dc=com
objectClass: top
objectClass: organizationalUnit
ou: people
```

```
# テストユーザー 1 (個人番号 01PLM01・Joe アカウント)
dn: uid=01PLM01,ou=people,dc=example,dc=com
objectClass: top
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
uid: 01PLM01
cn: Test User 01PLM01
sn: 01PLM01
mail: 01PLM01@plm-lab.local
userPassword: 01PLM01
```

```
# テストユーザー 2 (個人番号 01PLM02・Joe アカウント)
dn: uid=01PLM02,ou=people,dc=example,dc=com
objectClass: top
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
uid: 01PLM02
cn: Test User 01PLM02
sn: 01PLM02
mail: 01PLM02@plm-lab.local
userPassword: 01PLM02
```

```
# IdP 検索バインド用 (読取専用アカウント)
dn: uid=idp-reader,ou=people,dc=example,dc=com
objectClass: top
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
uid: idp-reader
cn: IdP Reader
sn: Reader
userPassword: idp-reader
```

実行前は未投入：LDIF タブ名が *LDIF ... (先頭に *) の間は編集集中で、サーバへは未送信。
右上の緑の ► (Execute LDIF) を押して初めて反映される。成功すると下部 **Modification Logs** に #1RESULT OK と各 add が記録され、左ツリーの dc=example,dc=com を Reload すると ou=people 配下が現れる。userPassword は Entry editor で「SSHA hashed password」と表示され、平文が自動ハッシュ化されたことを確認できる。

objectClass の階層：inetOrgPerson は organizationalPerson → person → top を継承するため、上記のように 4 つを併記します。person の必須属性 cn・sn を必ず与えます (Directory Studio の GUI 追加でも同様に求められます)。

GUI で投入する場合：LDAP Browser で dc=example,dc=com を右クリック → **New Entry - Create entry from scratch** で ou=people (organizationalUnit) を作成 → 続けて ou=people の下に objectClass inetOrgPerson のエントリを作成、RDN を uid=01PLM01、cn・sn・mail を入力し、最後に **New Attribute** で userPassword を追加。

7.6 動作確認 (検索バインド)

投入後、**検索バインド (idp-reader)** で**テストユーザーを引けるか**を確認します。これはフェーズ 5 で IdP が行う動作の先取り確認です。

方法1：Directory Studio で確認 - 新しい接続（例：plm-lab-idp-reader）を Port 10389 / Encryption No e nryption / Bind DN uid=idp-reader,ou=people,dc=example,dc=com / パスワード idp-reader で作成し、**Check Authentication** が successful になること。- LDAP Browser で ou=people を展開し、uid=01PLM01・01PLM02 が見えること（mail 属性に 01PLM01@plm-lab.local 等が入っていること）。

方法2：検索で確認（推奨） Directory Studio の検索機能で、検索ベース ou=people,dc=example,dc=com、フィルタ (uid=01PLM01) を実行し、1件返ることを確認。

#	確認内容	期待結果
1	ApacheDS サービス稼働	ApacheDS サービスが実行中、LDAP ポート 10389 で待受
2	管理者接続	uid=admin,ou=system / secret で接続可
3	ベース DN	既定パーティション dc=example,dc=com を使用
4	エントリ投入	ou=people,dc=example,dc=com 配下に 01PLM01 / 01PLM02 / idp-reader
5	検索バインド	uid=idp-reader,ou=people,dc=example,dc=com で bind（Check Authentication successful）し、(uid=01PLM01) が1件返る
6	mail 属性	01PLM01 の mail が 01PLM01@plm-lab.local
7	パスワード格納	userPassword が「SSHA hashed password」（平文が自動ハッシュ化）

フェーズ5のIdPは、このidp-readerでLDAPに検索バインドし、ログインユーザーのuidで認証、mailをNameID（emailAddress形式）の元として取得します。フェーズ5のIdP設定でそのまま使う値：**LDAP URL ldap://localhost:10389／検索ベース ou=people,dc=example,dc=com／検索バインド uid=idp-reader,ou=people,dc=example,dc=com／パスワード idp-reader／フィルタ (uid={user})／NameIDの元 mail。**

8. フェーズ4：OpenJDK + Tomcat（Windows サービス）

目的：Shibboleth IdP（フェーズ5）を載せる土台として、Tomcat 10.1をWindowsサービスとして稼働させる。OpenJDK（Temurin 17）は\$5.6で導入済みのため、本フェーズはTomcatに集中する。まずHTTP 8080で起動確認し、HTTPS 8443化はフェーズ6で行う。

前提：java -version が17を返し、JAVA_HOME=C:\opt\jdk-17が設定済みであること（\$5.6）。

IdP 5は**Tomcat 10.1系が必須**（9系や11は不可。Jakarta名前空間）。

8.1 Tomcat 10.1の入手と展開（zip）

1. Apache Tomcat 公式（<https://tomcat.apache.org/download-10.cgi>）から、**Tomcat 10.1.xの「Core」zip**（例 apache-tomcat-10.1.xx-windows-x64.zip または apache-tomcat-10.1.xx.zip）を入手しC:\lab\installersへ。
2. C:\opt\tomcatに展開（zip内のトップフォルダをC:\opt\tomcatに合わせる）。管理者PowerShell：

```
Expand-Archive -Path "C:\lab\installers\apache-tomcat-10.1.*.zip" -DestinationPath "C:\opt" -Force
# 展開されたフォルダ名を確認（例：apache-tomcat-10.1.56）
Get-ChildItem C:\opt | Where-Object Name -like "apache-tomcat-*"
# C:\opt\tomcat にリネーム（bin/conf/libなどが直下に来るように）
```

```
Rename-Item "C:\opt\apache-tomcat-10.1.xx" "C:\opt\tomcat"
# 確認: bin\startup.bat 等が見えること
Get-ChildItem C:\opt\tomcat\bin\startup.bat, C:\opt\tomcat\conf\server.xml
```

8.2 環境変数 (CATALINA_HOME) の設定

Tomcat の場所を示す CATALINA_HOME をシステム環境変数に設定します (サービス化・起動スクリプトが参照)。管理者 PowerShell :

```
[Environment]::SetEnvironmentVariable("CATALINA_HOME", "C:\opt\tomcat", "Machine")
```

設定後は新しい PowerShell / コマンドプロンプトを開き直す (既存セッションには反映されない)。JAVA_HOME は \$5.6 で設定済みのため、Tomcat のサービスも JVM を見つけられる。

8.3 Windows サービスとして登録 (service.bat)

zip 版には bin\service.bat (サービス登録スクリプト) が含まれます。管理者コマンドプロンプト (PowerShell ではなく cmd を推奨。service.bat はバッチのため) で実行します。

```
cd /d C:\opt\tomcat\bin
service.bat install
```

- 成功すると、サービス名 **Tomcat10** (既定。サービスの表示名は実機で「**Apache Tomcat 10.1 Tomcat10**」となる) が登録されます (service.bat install <名前> で任意名も可)。
- UAC が有効な場合、Tomcat10.exe 起動時に追加の権限を求められることがあります (管理者で実行していれば通過)。

JVM (Java) をサービスに確実に認識させる : サービスが JVM を見つけられないと起動に失敗します。GUI 設定ツール tomcat10w で確認・調整できます。

```
C:\opt\tomcat\bin\tomcat10w.exe //ES//Tomcat10
```

- 開いたダイアログの **Java** タブで、**Java Virtual Machine** が C:\opt\jdk-17\bin\server\jvm.dll (または ...\bin\jvm.dll) を指しているか確認。空欄や誤りがあれば設定。
- Startup type** を Automatic (自動起動) にしておくと、Windows 起動時に Tomcat も上がる (再起動堅牢性)。

service.bat install 実行時に JAVA_HOME が正しく通っていれば、JVM は自動設定されることが多いです。起動に失敗する場合はまずこの tomcat10w の Java タブを確認します。

8.4 起動と動作確認 (8080)

```
Start-Service Tomcat10
Get-Service Tomcat10          # Status が Running
# HTTP 応答 (既定コネクタは 8080)
Invoke-WebRequest http://localhost:8080/ -UseBasicParsing | Select-Object StatusCode
# 待受ポート確認
netstat -ano | findstr 8080
```

- ブラウザで http://localhost:8080/ を開き、Tomcat の初期ページが表示されれば成功。
- ログは C:\opt\tomcat\logs\ (catalina.*.log) で確認できます。

補足 (8080 の公開範囲) : 本構成では IdP へのブラウザアクセスはフェーズ 6 で HTTPS 8443 経由にするため、8080 は最終的に localhost 限定でも構いません (server.xml の HTTP

コネクタに address="127.0.0.1" を付けて絞れる)。フェーズ4の時点では既定（全インターフェース）で確認して問題ありません。

8.5 動作確認チェックリスト（フェーズ4）

#	確認内容	コマンド / 方法	期待結果
1	Java	java -version	17.x
2	Tomcat 展開	C:\opt\tomcat\bin\startup.bat 等の存在	直下に bin/conf/lib
3	環境変数	echo %CATALINA_HOME%（新規 cmd）	C:\opt\tomcat
4	サービス登録	Get-Service Tomcat10	サービスが存在
5	サービス稼働	Get-Service Tomcat10	Status: Running
6	HTTP 応答	http://localhost:8080/	初期ページ（200）
7	自動起動	tomcat10w の Startup type	Automatic

すべて確認できれば、フェーズ4は完了です。次はフェーズ5（Shibboleth IdP 5 の導入・LDAP 認証・emailAddress 形式 NameID の準備）です。

9. フェーズ5：Shibboleth IdP 5（テスト IdP）

目的：Windows 上の Tomcat に Shibboleth IdP 5 を導入し、ApacheDS（LDAP）で認証、mail 属性を **emailAddress 形式の NameID** の元として扱う準備までを行う。将来この IdP は Entra ID に差し替え可能（学習用テスト IdP）。

WSL 版との違い：install.sh→install.bat、Linux パス→Windows パス、systemctl restart →Tomcat サービス再起動。SAML の設計（LDAP 認証・NameID）は共通。NameID 形式は WSL 版の unspecified から **emailAddress** に変更し、元属性を uid→mail にする（顧客 Entra の形式に合わせる）。

前提：フェーズ4で Tomcat が 8080 で起動確認済み。フェーズ3の LDAP 引き継ぎ値（ldap://localhost:10389/ou=people,dc=example,dc=com/uid=idp-reader,ou=people,dc=example,dc=com/idp-reader/(uid={user})/NameID の元 mail）を使う。

9.1 IdP 5 の入手と展開

- Shibboleth 公式（<https://shibboleth.net/downloads/identity-provider/latest5/>）から **IdP 5 の zip**（Windows は zip 推奨。Windows 改行）を入手し C:\lab\installers へ。
 - latest5/ には**その時点の最新版だけ**が置かれる。実在版のファイル名を確認してから取得する（例：ブラウザで上記 URL を開き、shibboleth-identity-provider-5.x.y.zip を確認）。
- 任意の場所（例 C:\lab）に展開。展開先は導入後は不要。

```
Expand-Archive -Path "C:\lab\installers\shibboleth-identity-provider-5.*.zip" -DestinationPath "C:\lab" -Force
Get-ChildItem C:\lab | Where-Object Name -like "shibboleth-identity-provider-5*"
```

9.2 install.bat による導入

展開したディストリビューションフォルダに入り、bin\install.bat を**管理者コマンドプロンプト**で実行します（対話式）。


```
cd /d C:\lab\shibboleth-identity-provider-5.x.y
bin\install.bat
```

対話（プロンプト）での入力：

質問	入力値
Installation Directory (idp.home)	C:\opt\shibboleth-idp
Host Name	idp.plm-lab.local
SAML EntityID	https://idp.plm-lab.local/idp/shibboleth
Attribute Scope	plm-lab.local
Keystore Password	changeit
Sealer Password	changeit

- ・ 導入すると、署名・暗号化鍵(b) (idp-signing / idp-encryption / sealer 等) が自動生成され、C:\opt\shibboleth-idp\ に配置、war\idp.war がビルドされる。
- ・ RIPEMD-160 などの INFO ログは無害 (SAML では使わない) 。

install.bat が JAVA を見つけれない場合は、JAVA_HOME=C:\opt\jdk-17 (\$5.6) が設定され、新しいコマンドプロンプトで実行しているか確認する。

9.3 Tomcat への配置 (context フラグメント)

Tomcat に IdP の war を認識させる **context フラグメント** idp.xml を配置します。

1. フォルダを作成 (無ければ) : C:\opt\tomcat\conf\Catalina\localhost\
2. idp.xml を作成 (管理者権限。昇格 PowerShell 例) :

```
New-Item -ItemType Directory -Force -Path "C:\opt\tomcat\conf\Catalina\localhost" | Out-Null
Set-Content -Path "C:\opt\tomcat\conf\Catalina\localhost\idp.xml" -Encoding ASCII -Value '<context docBase="C:/opt/shibboleth-idp/war/idp.war" privileged="true" antiResourceLocking="false" swallowOutput="true" />'
```

docBase のパス区切りは / (スラッシュ) で記述する (Tomcat の context では Windows でも / が無難) 。

JSTL の追加 (必須) : IdP 5.2.3 では、状態ページ /idp/profile/status などが JSP+JSTL で描画されるため、**JSTL を war に追加しないと ClassNotFoundException: jakarta.servlet.jsp.jstl.core.Config – ServletException になる** (メタデータ /idp/shibboleth は JSTL 不要のため返るが、status は失敗する)。次の手順で **API と実装 (Glassfish) の 2 jar** を overlay に置き、build.bat で war を再ビルドする。

```
$lib = "C:\opt\shibboleth-idp\edit-webapp\WEB-INF\lib"
New-Item -ItemType Directory -Force -Path $lib | Out-Null
# JSTL API
Invoke-WebRequest -UseBasicParsing -Uri "https://repo1.maven.org/maven2/jakarta/servlet/jsp/jstl/jakarta.servlet.jsp.jstl-api/3.0.0/jakarta.servlet.jsp.jstl-api-3.0.0.jar" -OutFile "$lib\jakarta.servlet.jsp.jstl-api-3.0.0.jar"
# JSTL 実装 (Glassfish) ※ 実装が無いと Config クラスが見つからずエラーになる
Invoke-WebRequest -UseBasicParsing -Uri "https://repo1.maven.org/maven2/org/glassfish/web/jakarta.servlet.jsp.jstl/3.0.1/jakarta.servlet.jsp.jstl-3.0.1.jar" -OutFile "$lib\jakarta.servlet.jsp.jstl-3.0.1.jar"
```

```
cd /d C:\opt\shibboleth-idp\bin
build.bat
```

Rebuilding ...\\war\\idp.war / Overlay from ...\\edit-webapp / Creating war file ...\\war\\idp.war が出来れば成功。オフライン環境では別端末で 2 jar を取得し edit-webapp\\WEB-INF\\lib\\ にコピーしてから build.bat する。

9.4 LDAP 認証の設定 (ldap.properties)

C:\\opt\\shibboleth-idp\\conf\\ldap.properties を編集し、フェーズ 3 の値に合わせます (**平文 LDAP**・10389・dc=example,dc=com)。主な項目：

```
idp.authn.LDAP.authenticator      = bindSearchAuthenticator
idp.authn.LDAP.ldapURL            = ldap://localhost:10389
idp.authn.LDAP.useStartTLS        = false
idp.authn.LDAP.useSSL             = false
idp.authn.LDAP.baseDN             = ou=people,dc=example,dc=com
idp.authn.LDAP.subtreeSearch      = true
idp.authn.LDAP.userFilter         = (uid={user})
idp.authn.LDAP.bindDN             = uid=idp-reader,ou=people,dc=example,dc=com
idp.authn.LDAP.dnFormat           = uid=%s,ou=people,dc=example,dc=com
# 返す属性に mail を含める (NameID の元)
idp.authn.LDAP.returnAttributes   = mail,uid
```

- ・ **検索バインドのパスワード**は credentials\\secrets.properties に集約します (WSL 版と同様、二重定義による WARN を避ける)。C:\\opt\\shibboleth-idp\\credentials\\secrets.properties の該当行：

```
idp.authn.LDAP.bindDNCredential = idp-reader
```

ldap.properties 側に idp.authn.LDAP.bindDNCredential の行があれば **コメントアウト**して重複を避ける。

- ・ useStartTLS=false／useSSL=false のため、trustCertificates／trustStore 系の行は使われない (有効なら不要・コメントアウト可)。インストール直後は trustCertificates = %\\idp.home\\ /credentials/ldap-server.crt 等が有効になっていることがあり、存在しないファイルを指すため紛らわしい。平文 LDAP では両行をコメントアウトしておく。
- ・ 属性解決 (attribute-resolver) でも同じ LDAP を参照する場合は idp.attribute.resolver.LDAP.* を同様に設定するが、本書は NameID の元 mail を authn 側の returnAttributes で取得する構成を基本とする (フェーズ 10 で属性解放と NameID 生成を確定)。

9.5 emailAddress 形式 NameID の準備 (saml-nameid.xml)

mail 属性を **emailAddress 形式の NameID** として発行する定義を、C:\\opt\\shibboleth-idp\\conf\\saml-nameid.xml の <util:list id="shibboleth.SAML2NameIDGenerators"> の内側に追加します (詳細な解放設定はフェーズ 10 で仕上げる。ここでは生成器の準備まで)。

```
<bean parent="shibboleth.SAML2AttributeSourcedGenerator"
  p:omitQualifiers="true"
  p:format="urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress"
  p:attributeSourceIds="#{ 'mail' }" />
```

これにより、ログインユーザーの mail (例 01PLM01@plm-lab.local) が emailAddress 形式の NameID として発行できる状態になる。SP 側での REMOTE_USER へのマッピングはフェーズ 10 で行う。

注意（重複させない）：既存のコメントアウト例を有効化する場合、それとは別に同じ bean を追記しないこと。shibboleth.SAML2NameIDGenerators の中に同一の SAML2AttributeSourcedGenerator が2つ入っていると意図しない重複になる（1つだけにする）。SAML1 側（shibboleth.SAML1NameIDGenerators）は本構成では不要。

9.6 起動と動作確認

Restart-Service Tomcat10

Start-Sleep -Seconds 20

ステータス（テキストが返る）

Invoke-WebRequest http://localhost:8080/idp/profile/status -UseBasicParsing | Select-Object -ExpandProperty Content

メタデータ（<EntityDescriptor ...> が返る）

(Invoke-WebRequest http://localhost:8080/idp/shibboleth -UseBasicParsing).Content.Substring(0,300)

- C:\opt\shibboleth-idp\logs\idp-process.log に致命的 ERROR が無いこと（bindDNCredential の Duplicate WARN が出たら §9.4 の集約を確認）。
- install.bat 導入直後に bin\status.bat を実行すると、既定で http://localhost/idp/status（80 番）を見にいき「Connection refused」になることがあるが、これは想定どおり（本構成は 8080／のちに 8443）。確認は上記 8080 の URL で行う。

9.7 動作確認チェックリスト（フェーズ 5）

#	確認内容	期待結果
1	IdP 導入	C:\opt\shibboleth-idp\ に conf/credentials /metadata/war が存在
2	context 配置	C:\opt\tomcat\conf\Catalina\localhost\idp.xml が存在
2b	JSTL 追加	edit-webapp\WEB-INF\lib に JSTL の API+実装 2 jar があり build.bat 済み
3	Tomcat 起動	Tomcat10 が Running
4	ステータス	http://localhost:8080/idp/profile/status が環境情報を返す
5	メタデータ	http://localhost:8080/idp/shibboleth が<EntityDescriptor> を返す
6	LDAP 設定	ldap.properties が 10389／dc=example,dc=com／idp-reader、returnAttributes に mail
7	NameID 準備	saml-nameid.xml に emailAddress の SAML2AttributeSourcedGenerator（mail）
8	ログ	idp-process.log に致命的 ERROR・重複 WARN が無い

すべて確認できれば、フェーズ 5 は完了です。次はフェーズ 6（Tomcat を直接 HTTPS 8443 で公開）です。

10. フェーズ 6 : Tomcat 直 HTTPS (8443) 公開

目的: ブラウザから IdP へ **HTTPS (8443)** で到達できるようにする。WSL 版では前段に Apache を置いて TLS を終端したが、本構成では **Tomcat 自身に HTTPS コネクタを持たせて Apache を省略**する。証明書はフェーズ 2 で作成した idp.pfx (TLS/HTTPS サーバ証明書(a)・パスワード changeit) を使う。

WSL 版との違い: Apache HTTPD の導入・リバースプロキシ設定・mirrored ネットワーク・Listen 80/443 無効化がすべて不要。Tomcat の server.xml に SSL コネクタを 1 つ追加するだけ。

10.1 証明書 (idp.pfx) を Tomcat に配置

フェーズ 2 で作成した C:\lab\ca\idp.pfx を Tomcat の conf に配置します (管理者 PowerShell)。

```
Copy-Item "C:\lab\ca\idp.pfx" "C:\opt\tomcat\conf\idp.pfx" -Force
Get-Item "C:\opt\tomcat\conf\idp.pfx"
```

idp.pfx は idp.crt+idp.key+rootCA.crt を含む PKCS12 (SAN=idp.plm-lab.local) 。パスワードは changeit (\$3.1) 。

10.2 server.xml に HTTPS 8443 コネクタを追加

C:\opt\tomcat\conf\server.xml を管理者権限で編集します。既存の 8080 HTTP コネクタ (<Connector port="8080" .../>) の**近く**に、次の HTTPS 8443 コネクタを追加します (Tomcat 10.1 は **SSLHostConfig+Certificate** 方式) 。

```
<Connector port="8443" protocol="org.apache.coyote.http11.Http11NioProtocol"
    maxThreads="150" SSLEnabled="true" scheme="https" secure="true">
  <SSLHostConfig>
    <Certificate certificateKeystoreFile="conf/idp.pfx"
      certificateKeystorePassword="changeit"
      certificateKeystoreType="PKCS12"
      type="RSA" />
  </SSLHostConfig>
</Connector>
```

- certificateKeystoreFile="conf/idp.pfx" は CATALINA_BASE (=C:\opt\tomcat) からの相対で解決される。絶対パスにする場合は C:/opt/tomcat/conf/idp.pfx (**スラッシュ**) で書く。
- パスワードに <> を含む場合は XML エスケープが必要 (今回の changeit は不要) 。

10.3 8080 を localhost 限定にする (任意・推奨)

ブラウザは 8443 経由で IdP にアクセスするため、平文の 8080 は localhost だけに絞っておくと安全です。既存の 8080 コネクタに address="127.0.0.1" を追加します。

```
<Connector port="8080" protocol="HTTP/1.1"
  address="127.0.0.1"
  connectionTimeout="20000"
  redirectPort="8443" />
```

redirectPort を 8443 にしておくと、機密制約でリダイレクトが必要な場合に HTTPS へ誘導される。必須ではない。

10.4 反映と動作確認

Restart-Service Tomcat10

Start-Sleep -Seconds 20

8443 が待受

netstat -ano | findstr 8443

HTTPS で status (CA 検証済み。鍵マーク相当)

Invoke-WebRequest https://idp.plm-lab.local:8443/idp/profile/status -UseBasicParsing | Select-Object StatusCode

- Invoke-WebRequest が証明書エラーを出さずに 200 を返せば、rootCA 信頼 (\$6.5) と SAN (idp.plm-lab.local) が正しく効いています。
- ゲスト Windows の **ブラウザ** で https://idp.plm-lab.local:8443/idp/profile/status を開き、**鍵マーク (証明書警告なし)** で環境情報が表示されることを確認。
- 起動に失敗する場合は C:\opt\tomcat\logs\catalina.*.log を確認 (多くは certificateKeystoreFile のパス誤り、certificateKeystorePassword 不一致、certificateKeystoreType の指定漏れ)。

証明書警告が出る場合: rootCA が「信頼されたルート証明機関」に入っているか (\$6.5、certlm.msc)、アクセス URL のホスト名が idp.plm-lab.local (SAN と一致) か、hosts 解決 (127.0.0.1) を確認。

10.5 動作確認チェックリスト (フェーズ 6)

#	確認内容	期待結果
1	証明書配置	C:\opt\tomcat\conf\idp.pfx が存在
2	server.xml	8443 の SSLHostConfig+Certificate コネクタを追加
3	サービス起動	Tomcat10 が Running (8443 コネクタ起動失敗が無い)
4	待受	netstat で 8443 が LISTENING
5	HTTPS status	https://idp.plm-lab.local:8443/idp/profile/status が 200 (証明書エラーなし)
6	ブラウザ	鍵マークで status 表示 (警告なし)
7	8080	(任意) address="127.0.0.1" で localhost 限定

すべて確認できれば、フェーズ 6 は完了です。WSL 版で必要だった Apache 前段が不要になり、IdP がブラウザから HTTPS で到達可能になりました。次はフェーズ 7 (IIS の構築・SP の保護対象サイトと 443/TLS) です。

以降のフェーズ (\$11 フェーズ 7~\$15 フェーズ 11、および付録) は、フェーズごとに実機検証しながら順次追記します。